DEEP LEARNING THEORY–BASED PRACTICAL WORK

PHASE 2 · SOLUTION DESIGN

# Cross-Subject Motor Imagery EEG Classification using Deep Learning

Complete Methodology and Detailed Implementation Plan


| | |
|---|---|
| Team members | Ananya Gourishetty<br>Haarika D<br>Siddhanth Samal<br>Sathvik P |
| Group | Group 5 |
| Guide / faculty | Dr. Mohan Allam |
| Course | CSE4006 – Deep Learning |
| Department / institution | School of Computer Science and Engineering, VIT-AP University |


| | |
|---|---|
| Primary dataset | BCI Competition IV Dataset 2a |
| Review | Phase 2 review, 15 September 2026 |
| Planned implementation window | 16 September – 3 October 2026 |
| Planned final submission | 10 October 2026 |

# 1. Project Title

Cross-Subject Motor Imagery EEG Classification using a Hybrid Convolutional–Transformer Deep Learning Model.

The work develops a deep learning model that classifies four motor imagery tasks from electroencephalography, and evaluates it under a subject-independent protocol in which the test subject is not present in the training data. The model combines components that the surveyed literature has shown to be effective, while keeping the parameter count low enough to remain practical on modest hardware.

# 2. Problem Statement

Motor imagery is the act of imagining a movement without performing it. When a person imagines moving a limb, the sensorimotor cortex produces a measurable change in the mu band (approximately 8 to 13 Hz) and the beta band (approximately 13 to 30 Hz). This change is called event-related desynchronisation [13], and it appears on the side of the head opposite to the imagined limb. Electroencephalography records this activity from the scalp without surgery, which makes it the most widely used signal for practical brain–computer interfaces.

Building a classifier for these signals is difficult for several related reasons. The signal-to-noise ratio is low, because the cortical activity of interest is attenuated by the skull and scalp. The recordings are contaminated by eye movements, muscle activity and mains interference, all of which are often larger in amplitude than the signal being sought. The data are high-dimensional: a four-second trial recorded from 22 channels at 250 Hz contains more than twenty thousand samples, while a typical dataset provides only a few hundred labelled trials per subject. The signal is also non-stationary, so recordings made from the same person on different days do not have identical statistics.

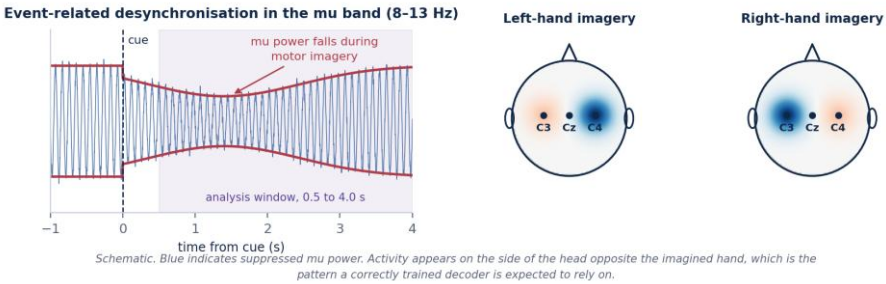



*Figure 2.1 — Motor imagery suppresses mu-band power in the sensorimotor cortex, on the side opposite the imagined hand. This is the physiological effect a decoder is expected to detect.*

The specific difficulty addressed in this project is variability between people. Skull thickness, cortical folding, electrode placement and the mental strategy a person adopts when imagining movement all differ from one individual to the next. As a result the statistical distribution of the recorded features shifts from subject to subject, and a model trained on one group of people often performs poorly when applied to someone new.

# 3. Motivation

A reliable motor imagery decoder would allow a person to issue commands to a computer or a device without any physical movement. The clearest application is assistive technology for people with severe motor impairment arising from conditions such as amyotrophic lateral sclerosis, spinal cord injury or brainstem stroke, for whom keyboards, touchscreens and speech interfaces may all be inaccessible. Related applications include neurorehabilitation, where detecting motor intent can be used to trigger assisted movement in patients relearning motor control, and hands-free interaction in settings where physical input is impractical.

For any of these applications the practical obstacle is the calibration session. If a system must be trained on several hours of labelled data from each new user before it becomes usable, it is difficult to deploy. Reducing that

requirement depends on the model generalising across people, which is why this project treats cross-subject performance as the central objective rather than as a secondary result.

## 4. Research Gap

The literature survey conducted in Phase 1 covered four representative deep learning architectures for motor imagery decoding: EEGNet [1], ATCNet [2], CTNet [3] and EEG Conformer [4]. Taken together they show a clear direction of development and a clear unresolved problem.




*Figure 4.1 — Development of the surveyed architectures, from a compact convolutional network towards convolutional–Transformer hybrids. Accuracies and parameter counts are as reported by the respective authors.*

### 4.1 Observations from the surveyed papers

- Cross-subject generalisation remains difficult. Accuracy is consistently lower when the test subject was not present in the training set.
- Zero-calibration and low-calibration operation is still an open problem. Most reported results assume that labelled data from the target subject are available.
- Convolutional models such as EEGNet are lightweight and train reliably on small datasets, but their receptive field limits how much long-range temporal structure they can represent.
- Transformer-based models such as CTNet and EEG Conformer capture global temporal relationships more directly, but they have substantially more parameters and are more sensitive to the limited size of EEG datasets.
- No single model in the surveyed set balances classification accuracy, model complexity and cross-subject generalisation at the same time.

### 4.2 Quantifying the gap

The size of the problem can be seen directly in the results reported by CTNet, which evaluates the same architecture under both a subject-specific and a subject-independent protocol.

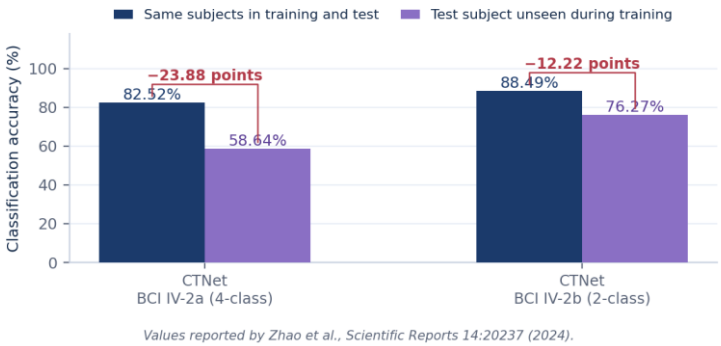



*Figure 4.2 — Accuracy reported by CTNet under subject-specific and subject-independent evaluation. The four-class task loses close to twenty-four percentage points when the test subject is unseen during training.*

A model that reaches 82.52 per cent on people it has already seen and 58.64 per cent on a new person is not yet suitable for deployment, because the second figure is the one that describes what a first-time user would experience. Reducing this difference, without a large increase in model size, is the gap this project addresses.

## 5. Project Objectives

1. To implement a reproducible preprocessing and evaluation pipeline for motor imagery EEG using BCI Competition IV Dataset 2a.
2. To implement EEGNet, ATCNet, CTNet and EEG Conformer as baseline models, and to evaluate them under a single identical protocol so that their results can be compared on equal terms.
3. To design and implement a hybrid convolutional–Transformer model that combines the convolutional feature extraction used in EEGNet with the attention mechanisms used in ATCNet, CTNet and EEG Conformer.
4. To train and evaluate all models using leave-one-subject-out cross-validation, so that every reported figure describes performance on a subject that was not seen during training.
5. To measure classification accuracy, Cohen's kappa, precision, recall and F1-score, and to report results for each subject as well as averaged across subjects.
6. To compare the parameter count and computational cost of the proposed model against the baselines.
7. To analyse whether the proposed combination improves cross-subject performance while remaining comparatively lightweight, and to report the outcome whether or not it does.

# 6. Proposed Solution

The proposed solution is a single hybrid model, referred to in this document as HCT-Net, or Hybrid Convolution–Transformer Network. Rather than selecting one of the surveyed architectures and applying it unchanged, the model brings together the specific component from each paper that addresses a specific weakness identified in Section 4. This section sets out the principles that guided the design, the alternatives that were considered and rejected, and the approach that was selected.

## 6.1 Design principles

Five principles were fixed before any architecture was drawn. Each follows from the research gap rather than from preference, and each is checkable.

| Principle | Statement | Follows from | How it is checked |
|---|---|---|---|
| **P1 — Cross-subject first** | Every design decision is judged by its effect on unseen subjects, not on within-subject accuracy | Section 4.2: the twenty-four point drop is the problem | All headline results use leave-one-subject-out |
| **P2 — Small by default** | Capacity is added only where a specific limitation requires it | Small dataset; deployment on modest hardware | Parameter count computed in Section 11.4 |
| **P3 — Every component justified** | No component is included because it appeared in a published paper | Combination without reason is not a design | Component study, experiment E4 |
| **P4 — One protocol for all models** | Baselines and the proposed model share preprocessing, folds and stopping rules | Published figures use different protocols and cannot be ranked | Single configuration file per experiment |
| **P5 — Honest reporting** | Negative and inconclusive outcomes are reported in the same detail as positive ones | The project is a study, not a demonstration | Result templates in Section 17.3 are fixed in advance |

*Table 6.1 — Design principles and the evidence each responds to.*

## 6.2 Candidate approaches considered

Six families of approach were considered before the hybrid was selected. They are set out below with the reason each was or was not adopted. The assessment is qualitative and reflects behaviour reported in the surveyed literature; it is not a measurement made by this project.

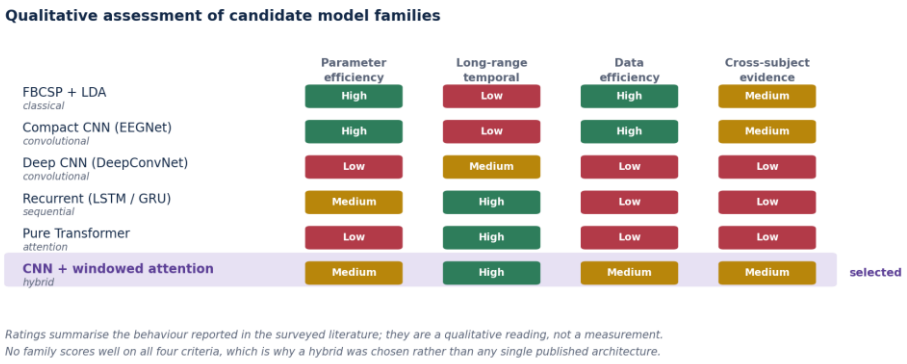



*Figure 6.1 — Qualitative assessment of candidate model families against the four criteria that follow from the research gap.*

| Approach | Why it was considered | Why it was not selected on its own |
|---|---|---|
| **FBCSP with linear discriminant analysis [9]** | Well understood, fast, no training instability, still competitive on small datasets | Feature extraction is fixed in advance and cannot adapt; retained as a control baseline rather than the main method |
| **Compact CNN (EEGNet alone)** | Very small and trains reliably; the natural lower bound on model size | Receptive field limits how much long-range temporal structure it can represent; retained as a baseline |
| **Deep CNN (DeepConvNet [8])** | Greater capacity than a compact CNN | Reported accuracy on this dataset is poor relative to its size, which suggests it overfits at this data scale |
| **Recurrent networks (LSTM, GRU)** | Designed for sequences and can carry information across a whole trial | Slow to train, difficult to stabilise on short noisy sequences, and not used by any of the four surveyed papers |
| **Pure Transformer on raw EEG** | Directly models relationships between any two time points | Very large parameter count and a strong data requirement; EEG datasets of this size do not support it without heavy augmentation |
| **Hybrid convolution with windowed attention** | Convolution supplies cheap spatial filtering; attention supplies long-range structure; windowing limits the cost of attention | Selected. It is the only family that is acceptable on all four criteria, although it is best on none |

*Table 6.2 — Candidate approaches and the reason each was accepted or set aside.*

## 6.3 The selected approach

The reasoning behind each component of the selected approach is as follows. The convolutional front end is taken from EEGNet, because depthwise and separable convolution provide spatial filtering comparable to classical common spatial patterns at a very small parameter cost, which directly serves principle P2. The temporal attention stage takes the sliding-window formulation used by ATCNet rather than attention over the full sequence, because windowing reduces both the parameter count and the tendency to overfit that is associated with unconstrained attention on small datasets. The use of a Transformer encoder [10] on top of a convolutional feature map follows CTNet and EEG Conformer, but with two encoder layers rather than the six used by CTNet, again for reasons of model size and data efficiency. Euclidean alignment is applied during preprocessing because it reduces the difference between subjects at zero parameter cost, which serves P1 and P2 at the same time. Finally, the training and evaluation procedure is subject-independent throughout, which none of the four papers adopts as its primary setting.

## 6.4 What is taken from the literature and what is proposed here

**Stated explicitly, so that the contribution is not overstated**

Taken from the literature: the depthwise and separable convolution block (EEGNet), the sliding-window attention formulation (ATCNet), the convolution-then-Transformer arrangement (CTNet and EEG Conformer), Euclidean alignment (He and Wu [7]), and segmentation-and-reconstruction augmentation (EEG Conformer and CTNet).

Proposed in this project: the particular combination of these components in one lightweight model, the reduced-depth attention configuration chosen for data efficiency, the subject-independent training and evaluation procedure applied uniformly to every model, and the component study that measures what each part contributes to cross-subject performance.

# 7. Proposed System Architecture

## 7.1 End-to-end flow

The complete system is organised as a sequence of stages, from the raw recording to a predicted motor imagery class.

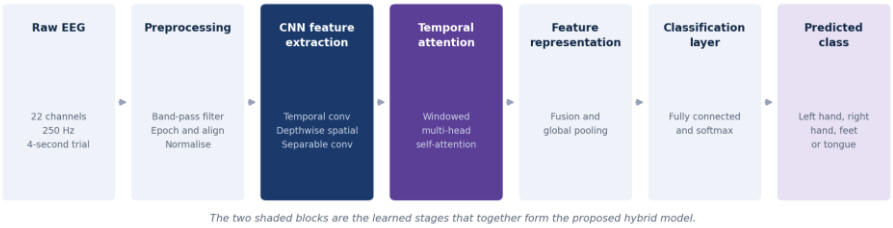



*Figure 7.1 — System architecture. The two shaded stages are the learned components that together form the proposed model; the remaining stages are fixed signal processing and output layers.*

| Stage | Function | Why it is required |
|---|---|---|
| **Raw EEG** | A trial recorded from 22 electrodes at 250 Hz | The input signal, as provided by the dataset |
| **Preprocessing** | Band-pass filtering, epoching, alignment and normalisation | Removes noise outside the frequency range of interest and places all subjects on a comparable scale |
| **CNN spatial–temporal feature extraction** | Temporal convolution, depthwise spatial convolution, separable convolution | Learns frequency-selective filters and spatial combinations of electrodes at low parameter cost |
| **Lightweight temporal attention** | Windowed multi-head self-attention over the convolutional feature sequence | Relates information from different parts of the trial, which convolution alone captures only within its receptive field |
| **Feature representation** | Fusion of window representations and global average pooling | Produces a single fixed-length vector describing the trial |
| **Classification layer** | Fully connected layer followed by softmax | Produces a probability for each of the four classes |
| **Predicted class** | Left hand, right hand, feet or tongue | The output of the system |

*Table 7.1 — Function of each stage in the system architecture.*

## 7.2 Data flow and tensor shapes

The table below traces a single trial through the system, so that the shape of the data at every interface is fixed before implementation begins. Batch dimensions are omitted.

| Point in the system | Shape | Data type | Produced by |
|---|---|---|---|
| **Raw recording, one session** | 25 × ~6.7 × $10^5$ | float32 | Dataset loader |
| **After channel selection** | 22 × ~6.7 × $10^5$ | float32 | data/loaders.py |
| **After band-pass filter** | 22 × ~6.7 × $10^5$ | float32 | preprocessing/filters.py |
| **After epoching** | 22 × 875 per trial | float32 | preprocessing/filters.py |
| **After artefact rejection** | 22 × 875, fewer trials | float32 | preprocessing/filters.py |
| **After Euclidean alignment** | 22 × 875 | float32 | preprocessing/align.py |
| **After standardisation** | 22 × 875 | float32 | preprocessing/filters.py |
| **Cached tensor on disk** | N × 22 × 875 plus N labels | float32, int64 | preprocessing cache |
| **After convolutional block** | 32 × 27 | float32 | models/hctnet.py |
| **After window segmentation** | 5 × 32 × 11 | float32 | models/hctnet.py |
| **After attention and fusion** | 32 | float32 | models/hctnet.py |
| **Model output** | 4 class logits | float32 | models/hctnet.py |

*Table 7.2 — Data flow with shapes at every interface. Fixing these in advance means each module can be written and tested independently.*

## 7.3 Software module structure and interfaces

The implementation is divided into modules with one responsibility each. The boundaries are chosen so that an error in one stage cannot silently affect another, and so that the expensive preprocessing step is performed once rather than once per model.

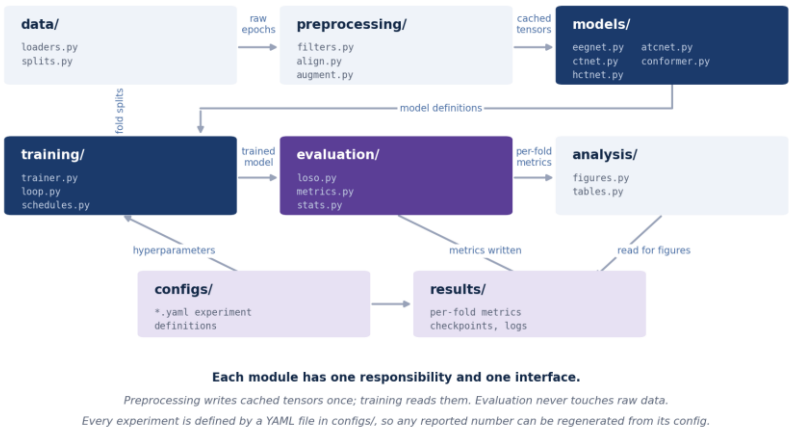



*Figure 7.2 — Software modules and the artefacts passed between them.*

| Module | Responsibility | Inputs | Outputs |
|---|---|---|---|
| **data/** | Load the dataset, select channels, define fold membership | Raw GDF files | Raw epochs, fold definitions |
| **preprocessing/** | Filter, epoch, reject, align, normalise, augment, cache | Raw epochs | Cached tensors on disk |
| **models/** | Define every architecture, including all baselines | Configuration | Model objects with a common interface |
| **training/** | Run one training job for one fold | Cached tensors, model, configuration | Trained weights, training log |
| **evaluation/** | Run the LOSO loop, compute metrics, run statistical tests | Trained weights, cached tensors | Per-fold metrics, test results |
| **analysis/** | Produce every figure and table in the final report | Per-fold metrics | Figures and tables |
| **configs/** | Define each experiment as a file | — | YAML configuration files |
| **results/** | Store metrics, checkpoints and logs | — | Files under version control except checkpoints |

*Table 7.3 — Module responsibilities and interfaces.*

## 7.4 What is fixed and what is learned

It is worth stating clearly which parts of the system have trainable parameters and which do not, because the distinction matters when the parameter count is compared against the baselines.

| Component | Trainable? | Note |
|---|---|---|
| **Band-pass filter** | No | Fixed fourth-order Butterworth, coefficients computed once |
| **Epoching and artefact rejection** | No | Deterministic given the thresholds |
| **Euclidean alignment** | No | One matrix per subject, computed from that subject's own unlabelled trials |
| **Standardisation** | No | Statistics computed from the training subjects |
| **Convolutional block** | Yes | 3,424 parameters |
| **Windowed attention block** | Yes | 17,440 parameters including positional encoding |
| **Classifier** | Yes | 132 parameters |

*Table 7.4 — Trainable and non-trainable components. Alignment reduces the difference between subjects without adding a single parameter, which is why it is applied in preprocessing rather than inside the network.*

# 8. Dataset Description

## 8.1 Why this dataset was selected

Five public motor imagery datasets were considered. BCI Competition IV Dataset 2a was selected as the primary dataset for the reasons set out below.

| Dataset | Subjects | Channels | Classes | Assessment |
|---|---|---|---|---|
| **BCI IV-2a** | 9 | 22 | 4 | Selected. Used by all four surveyed papers, so reproduced baselines can be checked against published values. Four classes make the task non-trivial. Small enough to run full leave-one-subject-out within the available time |
| **BCI IV-2b** | 9 | 3 | 2 | Only three electrodes and two classes. Useful as a secondary check but too easy to serve as the main task. Listed under future scope |
| **High Gamma Dataset** | 14 | 128 | 4 | More subjects and higher density, but 128 channels at 500 Hz makes preprocessing and training considerably heavier than the schedule allows |
| **PhysioNet Motor Movement/Imagery** | 109 | 64 | 4 | Far more subjects, which would strengthen a cross-subject claim, but it is not used by any surveyed paper, so no baseline comparison is possible |
| **SEED** | 15 | 62 | 3 | Excluded. This is an emotion recognition dataset with no motor imagery and no movement labels |

*Table 8.1 — Datasets considered, and the reason for the selection.*

## 8.2 Dataset properties

| Property | Value |
|---|---|
| **Dataset** | BCI Competition IV Dataset 2a |
| **Source** | Graz University of Technology, released for BCI Competition IV (2008) [5], [6] |
| **Number of subjects** | 9, labelled A01 to A09 |
| **EEG channels** | 22, arranged over sensorimotor cortex |
| **Additional channels** | 3 electrooculogram channels, used for artefact checking then discarded |
| **Motor imagery classes** | 4 — left hand, right hand, both feet, tongue |
| **Sampling rate** | 250 Hz |
| **Hardware band-pass** | 0.5 to 100 Hz, with a 50 Hz notch applied at acquisition |
| **Sessions per subject** | 2, recorded on different days |
| **Runs per session** | 6 |
| **Trials per run** | 48 (12 per class) |
| **Trials per session** | 288 (72 per class) |
| **Trials per subject** | 576 |
| **Total labelled trials** | 5,184 |
| **Class balance** | Exactly balanced by design |
| **Analysis window used here** | 0.5 to 4.0 s after the cue, giving 875 samples per trial |

*Table 8.2 — Properties of BCI Competition IV Dataset 2a.*

## 8.3 Trial structure and timing

Each trial follows a fixed timing protocol. Understanding it matters because the choice of analysis window is determined by it.

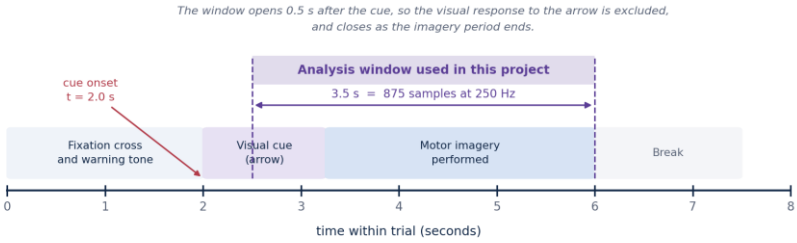



*Figure 8.1 — Timing of a single trial, and the position of the analysis window used in this project.*

A fixation cross appears at the start of the trial together with a short warning tone. At two seconds a visual cue in the form of an arrow indicates which movement to imagine, and remains on screen for 1.25 seconds. The subject performs the imagery until approximately six seconds, after which there is a short break of variable length.

The analysis window is taken from 0.5 seconds after the cue to 4.0 seconds after the cue, which corresponds to 2.5 to 6.0 seconds in absolute trial time. The first half-second after the cue is excluded deliberately, because it contains the visual evoked response to the arrow rather than motor imagery, and a model that learned to classify the arrow direction from the visual response would appear to work while decoding nothing about movement intent. The window closes at six seconds because the imagery period ends there. The resulting window is 3.5 seconds, or 875 samples at 250 Hz.

## 8.4 The four motor imagery classes

The four classes are distinguished by the region of sensorimotor cortex that becomes active during imagery. Hand imagery produces activity on the opposite side of the head, foot imagery produces activity near the midline, and tongue imagery produces activity in the lower lateral region.

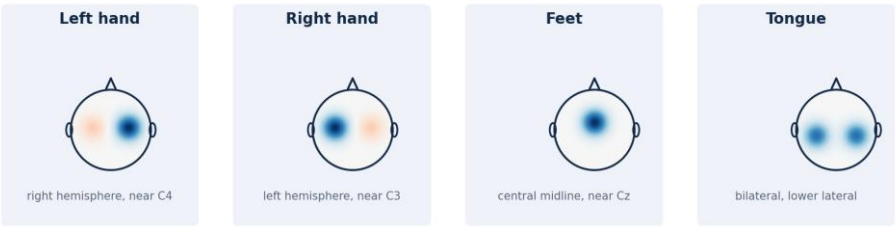



*Figure 8.2 — The four motor imagery classes and the scalp regions associated with each. The maps are schematic and indicate expected locations rather than measured values.*

## 8.5 Known characteristics and difficulties

- Feet and tongue imagery are harder to separate than left and right hand imagery, because both produce activity near the midline rather than on opposite sides. Confusion between these two classes is expected and will be visible in the confusion matrix.
- Performance varies considerably between subjects. Some individuals produce clear, consistent sensorimotor rhythms while others do not, a phenomenon sometimes described as brain–computer interface illiteracy. This is a property of the subjects, not of the model, and it is the reason per-subject results are reported.

- The two sessions were recorded on different days, so electrode placement and impedance differ between them. A model trained on session one and tested on session two therefore faces a distribution shift even within the same subject.
- With 5,184 trials in total, the dataset is small by deep learning standards. Regularisation and augmentation are necessary rather than optional, and models with large parameter counts are at real risk of overfitting.

### 8.6 Implications for the experimental design

Three consequences follow directly from the properties above and shape the rest of this document. First, leave-one-subject-out evaluation on nine subjects produces nine folds, which is enough to report a mean and a standard deviation and to apply a paired statistical test, but not enough to treat any single fold as reliable on its own. Second, because the classes are exactly balanced, accuracy is a meaningful headline metric, although macro-averaged precision, recall and F1 are still reported so that a model which ignores a difficult class cannot hide behind the average. Third, because the dataset is small, the parameter budget for the proposed model is set deliberately low, and this constraint is treated as a design requirement rather than as something to be optimised away.

# 9. Data Preprocessing

Preprocessing is applied identically to every model, so that any difference in results is attributable to the models rather than to differences in data handling. This section specifies each step exactly.

## 9.1 Step-by-step specification

| # | Operation | Parameters | Rationale |
|---|---|---|---|
| **1** | Load and select channels | Retain 22 EEG channels; discard 3 EOG channels after artefact checking | The EOG channels are not inputs to the model but are useful for identifying contaminated trials |
| **2** | Band-pass filter | 4 to 38 Hz, fourth-order Butterworth, applied forward and backward | Retains mu and beta rhythms; removes slow drift below 4 Hz and mains interference above 38 Hz. Applying the filter in both directions gives zero phase distortion, which preserves the temporal structure the attention stage relies on |
| **3** | Epoch | 0.5 to 4.0 s after cue, giving 875 samples | Excludes the visual evoked response to the cue; ends as the imagery period ends |
| **4** | Artefact rejection | Reject a trial if peak-to-peak amplitude exceeds 100 µV on any channel | Removes trials dominated by eye blinks or movement. The rejection rate is recorded per subject and reported |
| **5** | Euclidean alignment | Per subject and per session; see Section 9.2 | Reduces the difference between subjects before the network sees the data, at zero parameter cost |
| **6** | Standardisation | Per channel, using the mean and standard deviation of the training subjects only | Places all channels on a comparable scale without using information from the test subject |
| **7** | Augmentation | Segmentation and reconstruction, training split only; see Section 9.3 | Increases the effective number of training trials on a small dataset |
| **8** | Cache | Write tensors and labels to disk as compressed arrays | Preprocessing runs once, not once per model, which is what makes the experiment schedule feasible |

*Table 9.1 — Preprocessing specification. Every parameter is fixed here so that the pipeline can be implemented without further decisions.*

## 9.2 Euclidean alignment in detail

Euclidean alignment [7] is the step that does most of the work in reducing the difference between people, so it is worth setting out carefully what it does and why it works.

Every EEG trial can be summarised by its spatial covariance matrix, a 22 by 22 table in which each entry records how strongly the signals at two electrodes vary together. This matrix captures how the underlying cortical sources are mixed on their way to the scalp. That mixing is determined by physical properties of the individual: the thickness and conductivity of the skull, the shape and folding of the cortex, and the exact position in which the electrode cap was placed. None of these has anything to do with which movement the person is imagining, yet all of them shape the recorded signal. This is the main reason a model trained on one group of people transfers poorly to a new person.

The method proceeds in two steps. First, the covariance matrix of every trial belonging to one subject is computed and the average is taken. This average describes the typical way that subject's signals are mixed across electrodes, with the class-specific detail averaged out, because each class contributes roughly equally to the set. Second, every trial from that subject is multiplied by the inverse square root of that average matrix. The effect is that of a whitening transform: after it is applied, the average covariance of the subject's trials becomes the identity matrix.

The consequence is that all subjects arrive at the network sharing the same average spatial statistics. A model no longer has to spend capacity learning that one person's signals are generally larger at one electrode, or that two electrodes happen to be more correlated in one individual than another. What remains, and what the model is then free to learn, is the variation around that common average, which is where the class-discriminating information lives.

The inverse square root is computed through a symmetric eigendecomposition. The average covariance matrix is decomposed into eigenvectors and eigenvalues, each eigenvalue is raised to the power of minus one half, and the matrix is reassembled. One practical precaution is required: if two electrodes are nearly identical, for example because a channel was flat or duplicated, an eigenvalue can be extremely close to zero, and raising it to a negative power would produce a very large number that dominates the result. Small eigenvalues are therefore clamped to a small positive floor before the inverse is taken.

Two properties make this step particularly suitable for the present project. It is unsupervised, meaning it uses only the trials themselves and never their labels, so it can be applied to a new user who has not yet provided any labelled data at all. It also introduces no trainable parameters: the entire cost is one eigendecomposition of a 22 by 22 matrix per subject, which is negligible compared with a single forward pass of the network. It therefore improves cross-subject behaviour without working against the requirement that the model stay small.

Correctness is verified numerically rather than assumed. After alignment, the average covariance of each subject's trials is recomputed and compared against the identity matrix. If the two agree within a small tolerance, the implementation is behaving as intended. This check is written as a unit test and is the completion criterion for milestone M2 in Section 19.

### 9.3 Data augmentation

With 5,184 labelled trials in total, and roughly 4,032 available for training in any single fold, the dataset is small relative to the capacity of even a modest neural network. Augmentation is therefore a necessity rather than a refinement. The scheme used here is segmentation and reconstruction, following the procedure adopted by EEG Conformer and CTNet.

The idea is straightforward. Each trial is divided into a fixed number of equal intervals along the time axis; eight is used as the starting value. A new, synthetic trial is then assembled by taking the first interval from one real trial, the second interval from another, the third from a third, and so on, with every donor drawn from the pool of trials carrying the same class label. The result is a trial that never occurred in the recording session but which is composed entirely of genuine EEG belonging to that class.

The assumption underlying the method is that, within the analysis window, sub-intervals of trials of the same class are broadly interchangeable. This is reasonable for motor imagery, because the person is engaged in a sustained mental task throughout the window rather than producing a brief transient event, so any part of the interval carries the same kind of information. The recombination preserves the spatial structure of the signal exactly, since every interval retains all 22 channels together, and it preserves the spectral character of each interval. What it disrupts is the continuity of the signal at the boundaries between intervals, and the long-range temporal relationship between distant parts of a trial.

That disruption is the reason the number of segments matters. Dividing a trial into a small number of long intervals produces synthetic trials that stay close to real ones but adds little variety. Dividing it into many short intervals produces far more variety but leaves a signal that is discontinuous at every boundary and no longer resembles a genuine recording. Eight intervals, each roughly 0.44 seconds long, is the compromise the surveyed papers settle on, and it will be confirmed on the validation subject rather than tuned on the test subject.

Augmentation is applied to the training split only. Validation and test trials are always genuine recordings, because the purpose of those splits is to estimate performance on real data as a new user would supply it.

## 9.4 Preventing information leakage

**Leakage rules, applied without exception**

Normalisation statistics are computed from the training subjects only, never from the validation or test subject.

Augmentation is applied to the training split only.

Hyperparameters, including the early stopping point, are selected using the validation subject only.

The test subject contributes unlabelled trials to the Euclidean alignment step and nothing else. This is the same information a real deployment would have, since a new user can record unlabelled data before any labels exist.

Any departure from these rules invalidates the cross-subject result, and would need to be reported as a limitation rather than corrected silently.

## 10. Exploratory Data Analysis Plan

Exploratory analysis is carried out before any model is trained. Its purpose is threefold: to confirm that the data are as the dataset description claims, to establish that the preprocessing pipeline behaves correctly, and to document the variability between subjects that the rest of the project attempts to address. Each analysis has a stated expectation, so that a result which contradicts it is recognised as a problem rather than accepted.

| Analysis | What it shows | Expected result |
|---|---|---|
| **Trial counts and class balance per subject** | Whether any subject or class is under-represented after artefact rejection | 576 trials per subject and 144 per class before rejection |
| **Artefact rejection rate per subject** | Data quality differences between subjects | Under roughly 15 per cent for most subjects |
| **Power spectral density per channel and class** | Whether mu and beta activity are present in the expected bands | Clear peaks near 10 Hz and 20 Hz over sensorimotor channels |
| **Time–frequency maps at C3, Cz and C4** | Whether event-related desynchronisation appears after the cue | Power reduction in the mu band from roughly 0.5 s after the cue |
| **Topographic maps averaged by class** | Whether hand imagery produces activity on the opposite side of the head | Left-hand imagery strongest near C4 and right-hand imagery near C3 |
| **Covariance distance between subjects** | How far apart subjects are before and after alignment | A clear reduction in pairwise distance after alignment |
| **Low-dimensional projection of trial features** | Whether trials cluster by class or by subject | Clustering by subject before alignment, and less of it afterwards |

*Table 10.1 — Planned exploratory analyses and the outcome expected from each.*

Each analysis carries an expectation, so that a result which contradicts it is recognised as a problem rather than accepted. A mismatch is investigated and resolved before any model is trained, since an error in the data that goes unnoticed at this point would affect every result that follows and would be very difficult to trace later. The figures produced by this stage are kept and included in the final report, because they document that the pipeline was verified rather than assumed to be correct.

# 11. Proposed Deep Learning Architecture

## 11.1 Design constraints

The architecture is constrained before it is designed. The constraints below follow from the design principles in Section 6.1 and from the dataset properties in Section 8.

| Constraint | Value | Reason |
|---|---|---|
| **Trainable parameters** | Below 50,000 | A ceiling near ShallowConvNet's 44,644 and far below ATCNet's 113,732; a small model is less able to memorise subject-specific detail |
| **Attention span** | Windowed, not global | Global attention over 27 time steps is affordable but unconstrained; windowing imposes a locality prior that suits short EEG trials |
| **Encoder depth** | 2 layers | CTNet uses 6; with fewer than 6,000 trials in total, additional depth is more likely to fit subject-specific structure than shared structure |
| **Input length** | 875 samples | Fixed by the analysis window in Section 8.3 |
| **Output** | 4 classes | Fixed by the dataset |

*Table 11.1 — Architectural constraints fixed in advance.*

## 11.2 Block diagram




*Figure 11.1 — Block diagram of the proposed model. Tensor shapes are given for BCI Competition IV Dataset 2a, and the badges indicate which surveyed paper each component is adapted from.*

## 11.3 Layer-by-layer specification

The configuration below is written for 22 channels and 875 samples per trial.

| # | Layer | Configuration | Output shape |
|---|---|---|---|
| **1** | Temporal convolution | F1 = 16 filters, kernel (1, 64), padding same, no bias | 16 × 22 × 875 |
| **2** | Batch normalisation | — | 16 × 22 × 875 |
| **3** | Depthwise spatial convolution | D = 2, kernel (22, 1), max-norm constraint 1.0, no bias | 32 × 1 × 875 |
| **4** | Batch normalisation, ELU | — | 32 × 1 × 875 |
| **5** | Average pooling, dropout | Pool (1, 4), dropout 0.25 | 32 × 1 × 218 |
| **6** | Separable convolution | F2 = 32 filters, kernel (1, 16), depthwise then pointwise | 32 × 1 × 218 |
| **7** | Batch normalisation, ELU | — | 32 × 1 × 218 |
| **8** | Average pooling, dropout | Pool (1, 8), dropout 0.25 | 32 × 1 × 27 |
| **9** | Window segmentation | 5 overlapping windows of length 11, stride 4 | 5 × 32 × 11 |
| **10** | Positional encoding | Learned, shape (11, 32), added to each window | 5 × 32 × 11 |
| **11** | Transformer encoder | 2 layers, 2 heads, model dimension 32, feed-forward 64, GELU | 5 × 32 × 11 |
| **12** | Fusion and global average pooling | Mean over windows, then over time | 32 |
| **13** | Fully connected, softmax | 32 → 4 | 4 |

*Table 11.2 — Layer specification of the proposed model.*

## 11.4 Parameter budget

The parameter count is calculated below rather than estimated, so that the claim that the model is lightweight can be checked before implementation. The count assumes convolutions without bias terms, as in EEGNet, and a feed-forward expansion factor of two inside the encoder.

| Component | Calculation | Parameters |
|---|---|---|
| **Temporal convolution** | 16 × 1 × 1 × 64 | 1,024 |
| **Batch normalisation 1** | 2 × 16 | 32 |
| **Depthwise spatial convolution** | 16 × 2 × 22 × 1 | 704 |
| **Batch normalisation 2** | 2 × 32 | 64 |
| **Separable convolution, depthwise** | 32 × 1 × 16 | 512 |
| **Separable convolution, pointwise** | 32 × 32 | 1,024 |
| **Batch normalisation 3** | 2 × 32 | 64 |
| **Convolutional block subtotal** | | 3,424 |
| **Attention projections, per layer** | 4 × (32 × 32 + 32) | 4,224 |
| **Layer normalisation, per layer** | 2 × 2 × 32 | 128 |
| **Feed-forward, per layer** | (32 × 64 + 64) + (64 × 32 + 32) | 4,192 |
| **Encoder layer subtotal** | 4,224 + 128 + 4,192 | 8,544 |
| **Encoder, 2 layers** | 2 × 8,544 | 17,088 |
| **Learned positional encoding** | 11 × 32 | 352 |
| **Classifier** | 32 × 4 + 4 | 132 |
| **Total** | 3,424 + 17,088 + 352 + 132 | 20,996 |

*Table 11.3 — Calculated parameter budget for the proposed model.*

| Model | At 875 samples | At 1,125 samples | Relative to the proposed model |
|---|---|---|---|
| **EEGNet** | 3,188 | 3,700 | 0.15 × |
| **EEG-TCNet** | 4,304 | 4,304 | 0.20 × |
| **HCT-Net (proposed)** | 20,996 | — | 1.00 × |
| **ShallowConvNet** | 44,644 | 47,364 | 2.13 × |
| **ATCNet** | 113,732 | 113,732 | 5.42 × |
| **CTNet** | 152,364 | 153,004 | 7.26 × |
| **DeepConvNet** | 282,079 | 284,479 | 13.43 × |
| **EEG Conformer** | 697,412 | 871,492 | 33.22 × |

*Table 11.4 — Parameter count in context, for 22 channels and four classes, computed with the braindecode 1.8.1 reference implementations at this project's 875-sample input and at the 1,125-sample input used by the ATCNet reference implementation. The ratio column compares like with like, at 875 samples. For models whose classifier sizes itself from the input, the count depends on window length: EEGNet has 3,188 parameters at 875 samples and 3,700 at 1,125, and the figure of 2,548 often quoted for EEGNet corresponds to a window of about 2.2 s. An earlier version of this table gave 4,096 for EEG-TCNet and 553,654 for DeepConvNet; neither matches the reference implementation at any input length, and both have been replaced. The proposed model is roughly one fifth the size of ATCNet and comfortably inside the 50,000 budget.*

## 11.5 Purpose of each component

| Component | Adapted from | What it does | Why it is included |
|---|---|---|---|
| **Temporal convolution** | EEGNet | Learns band-pass filters directly from the data | Avoids fixing the frequency bands in advance |
| **Depthwise spatial convolution** | EEGNet | Learns a spatial filter for each temporal filter | Spatial filtering comparable to common spatial patterns at very low parameter cost |
| **Separable convolution** | EEGNet | Summarises the temporal pattern of each feature map | Reduces dimensionality before the attention stage |
| **Window segmentation** | ATCNet | Divides the feature sequence into overlapping windows | Limits the span over which attention operates, reducing parameters and overfitting |
| **Multi-head self-attention** | CTNet, EEG Conformer | Relates information from different parts of the trial | Addresses the limited receptive field of the convolutional stage |
| **Euclidean alignment** | Transfer learning literature | Standardises the spatial covariance of each subject | Reduces distribution shift between subjects without adding parameters |
| **Segmentation and reconstruction** | EEG Conformer, CTNet | Generates additional training trials | Compensates for the small number of labelled trials |

*Table 11.5 — Components of the proposed model, their origin, and the reason for including each.*

## 11.6 Why two encoder layers rather than six

CTNet uses a six-layer Transformer encoder and reports strong within-subject accuracy. This project uses two. The reasoning is that the dataset contains 5,184 labelled trials in total, and under leave-one-subject-out only about 4,032 of those are available for training in any fold. A six-layer encoder at this model dimension would triple the attention parameter count, from 17,088 to 51,264, which alone would exceed the budget set in Table 11.1 before the convolutional block is counted.

There is also a substantive reason rather than merely a budgetary one. Additional attention capacity gives the model more freedom to fit structure that is specific to the training subjects, and that structure does not transfer to a new person. The gap documented in Figure 4.2 is precisely the symptom of a model fitting subject-specific detail. Reducing capacity is therefore a plausible response to the problem, not only a concession to hardware.

Whether it is the correct response is an empirical question, and it is tested directly as experiment E5 in Section 17.2.

## 11.7 How a trial passes through the model

This section follows a single trial from input to prediction and explains what each stage is doing to the signal, rather than restating the layer table.

The input is a preprocessed trial of 22 channels by 875 samples. The first operation is a convolution along time only, with sixteen separate filters each spanning 64 samples. At 250 Hz, 64 samples is about a quarter of a second, which covers roughly two to three cycles of a mu rhythm. Each filter is therefore learning to respond to a particular frequency band, and because the filters are learned rather than fixed, the network chooses its own bands instead of being restricted to the mu and beta ranges specified in advance. At this point the trial has become sixteen filtered copies of itself, one per band, each still carrying all 22 channels.

The second operation is a depthwise convolution across channels. Its kernel spans the full height of 22 electrodes and a single time sample, so it forms weighted combinations of electrodes at each instant. This is the same idea as a classical common spatial pattern filter, except that it is learned jointly with everything else. The depth multiplier of two means each of the sixteen frequency bands receives two independent spatial filters, giving 32 feature maps in total, and it collapses the channel dimension to one. A max-norm constraint is applied to these weights so that no single electrode combination is allowed to grow without limit and dominate the representation.

Two rounds of pooling then reduce the time axis. The first averages over groups of four samples, taking the length from 875 down to 218, and the second averages over groups of eight, taking it down to 27. The reason for reducing so aggressively is that the information carried by a mu rhythm lies in how its power changes over hundreds of milliseconds, not in the position of individual peaks, so averaging discards timing detail the classifier does not need while cutting the cost of everything that follows. Between the two pooling stages is a separable convolution, which first summarises the temporal pattern within each feature map independently and then mixes the maps together, a factorisation that achieves the same effect as a full convolution at a fraction of the parameter cost.

What leaves the convolutional stage is a sequence of 27 positions, each described by 32 features. This sequence is where the attention stage operates. Rather than allowing every position to attend to every other, the sequence is cut into five windows of eleven positions each, with a stride of four so that neighbouring windows overlap. The overlap matters: without it a hard boundary could fall in the middle of an informative interval and split it between two windows that never see each other. Windowing also constrains what the attention mechanism can learn, which on a dataset of this size is an advantage rather than a limitation.

A learned positional encoding is added to each window before attention is applied, because self-attention is otherwise indifferent to order and would treat a window as an unordered set. Inside each window, two attention heads compute how strongly every position should influence every other, and a small feed-forward network transforms the result. This repeats over two encoder layers. The practical effect is that a feature at one moment in the trial can be interpreted in the light of what is happening at another moment within the same window, which pure convolution cannot do beyond the span of its kernel.

Finally the five window representations are averaged together, and the time axis within them is averaged as well, producing a single vector of 32 numbers that describes the whole trial. A fully connected layer maps that vector to four values, one per motor imagery class, and a softmax turns them into probabilities. The predicted class is the one with the highest probability.

# 12. Detailed Methodology

This section sets out the complete method, stage by stage. Each stage states what it receives, what it does, what it produces, and the check that must pass before the next stage begins. The intention is that the project could be carried out by following this section alone.




*Figure 12.1 — The methodology as a sequence of stages.*

## 12.1 Stage 1 — Data acquisition and verification

The dataset is downloaded programmatically through MOABB [12], which handles the file format and the standard channel naming. Before anything else is done, the loaded data are verified against the published dataset description, because every later result depends on the data being what it is assumed to be.

| Receives | Does | Produces |
|---|---|---|
| Nothing; downloads from source | Download all 9 subjects, both sessions. Verify channel count, sampling rate, trial count and class balance against Table 8.2 | Raw epochs held in memory or on disk |

Any mismatch at this point is resolved before continuing. A silent discrepancy here, such as a mislabelled class or a missing run, would propagate into every subsequent result and would be extremely difficult to detect later.

## 12.2 Stage 2 — Preprocessing

The pipeline specified in Section 9 is implemented and applied. The output is written to disk as cached tensors, one file per subject per session, so that the operation is performed once for the whole project rather than repeated for every model and every fold.

| Receives | Does | Produces |
|---|---|---|
| Raw epochs from Stage 1 | Filter, epoch, reject artefacts, align, standardise, cache | Cached tensors of shape N × 22 × 875 with labels, per subject |

## 12.3 Stage 3 — Exploratory analysis

The analyses listed in Table 10.1 are carried out on the cached data. This stage is a verification step as much as an exploratory one: several of the checks in that table would reveal a preprocessing error that the numeric assertions in Stage 2 would not catch. For example, a mirrored channel montage would produce correct covariance statistics but reversed topographic maps.

| Receives | Does | Produces |
|---|---|---|
| Cached tensors | Run every analysis in Table 10.1 and record the result | A set of figures, retained for the final report |

### 12.4 Stage 4 — Baseline implementation and reproduction

The four surveyed architectures and the classical control are implemented, using published reference implementations wherever they are available. Each is then trained and tested within subject, session one to session two, and the resulting accuracy is compared against the value published by its authors.

This reproduction step is not a result in itself and will not be presented as one. Its purpose is diagnostic. If a reproduced within-subject accuracy differs substantially from the published value, then either the implementation or the preprocessing is wrong, and the cross-subject figures that follow could not be trusted. Establishing this before the main experiments is the reason a dedicated milestone in Section 19 is set aside for it.

| Receives | Does | Produces |
|---|---|---|
| Cached tensors, model definitions | Train each baseline within subject; compare to published accuracy | A reproduction table with published and reproduced figures side by side |

### 12.5 Stage 5 — Proposed model implementation

The proposed model is implemented according to Table 11.2 and the description in Section 11.7. Three checks are applied before it is trained on real folds. First, tensor shapes are asserted at every layer against Table 11.2. Second, the parameter count is printed and compared against Table 11.3. Third, the model is trained deliberately to overfit a single batch of a few trials; if it cannot reach near-zero loss on data it has seen repeatedly, there is a defect in the implementation that no amount of tuning will fix.

| Receives | Does | Produces |
|---|---|---|
| Specification in Section 11 | Implement, assert shapes, count parameters, overfit one batch | A model class with the same interface as the baselines |

### 12.6 Stage 6 — Training

All models are trained using the configuration set out in Section 13, with the same optimiser, the same learning-rate schedule and the same stopping rule, so that any difference in results reflects the models rather than the way they were trained.

Training begins by loading the cached tensors for the seven training subjects and applying augmentation to them. The validation subject's trials are loaded without augmentation, because their purpose is to estimate how the model behaves on genuine data. The optimiser is Adam with an initial learning rate of 0.001, and the learning rate follows a cosine schedule that decays smoothly over the maximum epoch count, allowing large adjustments early in training and fine adjustments towards the end.

Within each epoch the training trials are shuffled and presented in batches of 64. For each batch the model produces class scores, a cross-entropy loss with label smoothing is computed against the true labels, and the gradients of that loss are propagated back through the network to update the weights. Immediately after each update the max-norm constraint is reapplied to the depthwise spatial convolution, so that no spatial filter can grow beyond the permitted magnitude between updates.

At the end of every epoch the model is switched to evaluation mode, which disables dropout and freezes the batch normalisation statistics, and its accuracy on the validation subject is measured. If that accuracy is the best seen so far, the current weights are saved and the patience counter is reset. If it is not, the counter increases, and training

stops once it reaches the patience limit. Stopping this way means the model is never allowed to keep improving on the training subjects at the expense of a subject it has not been trained on, which is the behaviour that produces the cross-subject gap described in Section 4.

What is returned is not the model as it stood at the final epoch but the checkpoint with the best validation accuracy, since the epochs after that point were by definition moving in the wrong direction.

### 12.7 Stage 7 — Cross-subject evaluation

The leave-one-subject-out loop is the outermost structure of the experiments. It is written once and reused by every model, so that no model can accidentally be evaluated under a different protocol from another.

The loop runs nine times, once for each subject. On each pass one subject is designated the test subject and a second, taken as the next subject in order, is designated the validation subject. The remaining seven form the training set. Because the validation role rotates along with the test role, no subject occupies the same position twice, and every subject serves as the unseen test subject exactly once.

At the start of each fold the random seeds are reset and a completely fresh model is constructed. This matters: carrying weights over from a previous fold would mean the model had already seen the subject it is about to be tested on, which would invalidate the entire result. Training then proceeds as described in Stage 6, using only the seven training subjects and the one validation subject.

Once training has finished, the held-out subject's trials are loaded. These are never augmented and never contributed anything to the choice of weights or hyperparameters. The trained model produces a prediction for each of them, and accuracy, Cohen's kappa, macro-averaged precision, recall and F1, and the confusion matrix are computed and recorded against that subject's identifier.

The loop therefore produces nine independent measurements rather than a single number, and it is those nine that are reported, both individually and as a mean with a standard deviation. Each configuration is run with three random seeds and the per-fold results averaged, so that a difference between two models is not an artefact of a single fortunate initialisation.

### 12.8 Stage 8 — Statistical analysis

Comparing two mean accuracies is not sufficient to claim that one model is better than another, particularly with only nine folds. Each pairwise comparison uses the Wilcoxon signed-rank test [15], applied to the nine paired per-subject accuracies. The test is non-parametric, which matters because per-subject accuracy is not normally distributed across subjects, and it is paired, which matters because the same nine subjects are used for every model.

Where multiple comparisons are made against the same baseline, the significance level is adjusted accordingly, and both the adjusted and unadjusted results are reported.

### 12.9 Stage 9 — Component study

The component study removes one part of the proposed model at a time and repeats the full leave-one-subject-out evaluation, so that the contribution of each part can be attributed rather than assumed. The configurations are listed in Table 17.2.

This stage is what distinguishes a designed model from an assembled one. Without it, a combination of published components is only a combination; with it, the project can state which parts of the combination mattered for cross-subject performance and which did not.

### 12.10 The complete method in summary

Taken together, the nine stages form a single sequence with three distinct phases.

The first phase is performed once for the whole project. The dataset is downloaded and verified against its published description, the preprocessing pipeline is applied and its output cached to disk, and the exploratory

analyses are run to confirm that the cached data behave as the physiology predicts. Nothing beyond this point is attempted until these checks pass, because every subsequent result depends on them.

The second phase is repeated for each of the six models. Each is first reproduced within subject and checked against its published accuracy, which establishes that the implementation and the pipeline are sound. Each is then trained and evaluated under the leave-one-subject-out protocol, producing nine per-subject measurements per model per seed. Because the folds, the preprocessing and the training configuration are identical for every model, the resulting figures are directly comparable, which is not true of the published numbers the models were originally reported with.

The third phase interprets what was measured. The proposed model is compared against each baseline using a paired statistical test across the nine subjects, and the component study repeats the evaluation with individual parts of the proposed model removed, so that any difference observed in the second phase can be attributed to a specific design choice rather than to the combination as a whole. The tables and figures required for the review are then generated from the recorded per-fold metrics rather than from any separate calculation, so that what is presented is exactly what was measured.

# 13. Training Strategy

All models are trained with the same optimiser settings and the same stopping rule, so that the comparison between them is fair. The settings below are the starting configuration; any change made during tuning is applied to every model and recorded in the configuration file for that experiment.

| Setting | Value | Reason |
|---|---|---|
| **Loss function** | Cross-entropy with label smoothing of 0.1 | Label smoothing reduces overconfidence, which matters on a small dataset where a model can become certain about patterns it has seen only a few times |
| **Optimiser** | Adam, initial learning rate 0.001 | Adam is the optimiser used by all four surveyed papers; changing it would make comparison with their results harder to interpret |
| **Learning rate schedule** | Cosine annealing over the maximum epoch count | Allows large steps early and fine adjustment late without a hand-tuned step schedule |
| **Batch size** | 64 | Fits comfortably in the memory of the available GPUs and gives stable batch normalisation statistics |
| **Maximum epochs** | 500 | Generous upper bound; early stopping is expected to trigger first |
| **Early stopping** | Patience of 50 epochs on validation subject accuracy | Stops before the model begins fitting the training subjects specifically |
| **Dropout** | 0.25 after each pooling stage | The value EEGNet's authors use for cross-subject classification; they use 0.5 within subject |
| **Weight constraint** | Max-norm of 1.0 on the depthwise convolution | Used by EEGNet to keep individual spatial filters from dominating |
| **Weight decay** | 0.0001 | Mild additional regularisation, applied uniformly |
| **Data augmentation** | Segmentation and reconstruction, training split only | Increases the effective number of training trials; never applied to validation or test data |
| **Random seeds** | Three per configuration, results averaged | A single seed can produce a difference between models that is initialisation noise rather than a real effect |
| **Mixed precision** | Enabled where supported | Reduces training time by roughly a third, which matters given the number of runs |

*Table 13.1 — Training configuration, with the reason for each value.*

## 13.1 Determinism and reproducibility of training

Every run fixes the seeds of Python, NumPy and PyTorch, and records them alongside the result. Deterministic algorithms are enabled where PyTorch provides them. Complete bit-level determinism on a GPU is not always achievable, so the project does not claim it; what is claimed and checked is that re-running a configuration with the same seed reproduces the reported accuracy to within a small tolerance, and that tolerance is reported.

## 13.2 An optional variant

If the schedule allows, one additional variant will be tested, in which an auxiliary branch attempts to predict which subject produced each trial and its gradient is reversed before reaching the feature extractor, so that the encoder is discouraged from retaining subject-identifying information. This is treated as an experimental extension rather than part of the main model, because it introduces a training-time weighting that must itself be tuned, and tuning it properly would consume time that the core experiments need. If it is not reached, it is recorded as future work rather than quietly dropped.

# 14. Cross-Subject Evaluation using Leave-One-Subject-Out

Leave-one-subject-out cross-validation is the evaluation protocol used throughout this project. In each fold, one subject is removed entirely from the training data and is used only for testing. This directly measures what a new user would experience.

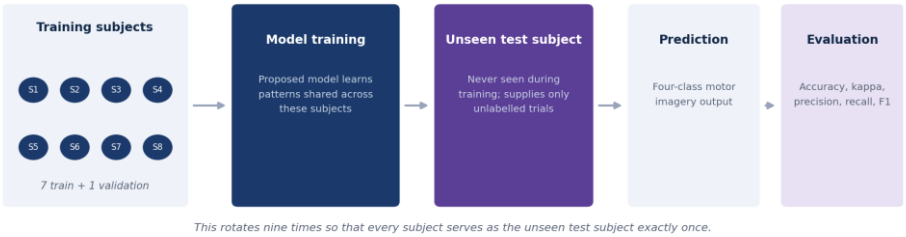



*Figure 14.1 — The leave-one-subject-out concept. The model is trained on a group of subjects and evaluated on a subject it has never seen.*

With nine subjects the procedure produces nine folds. In each fold, seven subjects are used for training, one is held out for validation and one is held out for testing. The reported figure is the mean and standard deviation across the nine folds.

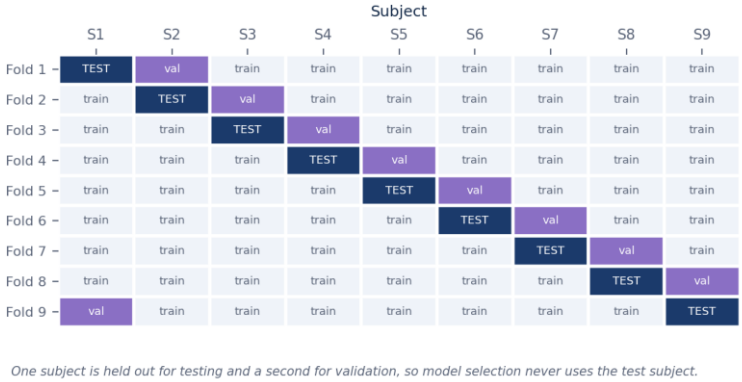



*Figure 14.2 — Fold structure. Every subject serves as the test subject exactly once, and a separate subject is used for validation in each fold.*

## 14.1 Why a validation subject rather than a validation split

A separate validation subject is used rather than a random split of the training subjects' trials. If validation trials came from subjects who also appear in the training set, the early stopping point and any tuned hyperparameter would be chosen to favour within-subject fit, which is exactly the behaviour the protocol is intended to avoid. Holding out a whole subject costs one subject's worth of training data in each fold, and that cost is accepted deliberately.

## 14.2 The within-subject reference protocol

For reference, each model is also trained and tested within subject, using session one for training and session two for testing. This is not a headline result. Its purpose is to check the pipeline: if the reproduced within-subject accuracy of a baseline differs substantially from the published value, then something in the implementation or preprocessing is wrong, and the cross-subject figures cannot be trusted either. Both protocols are reported so that the difference between them can be quantified for every model, which is itself informative.

## 15. Baseline Models

Four baselines are implemented, corresponding to the four papers in the literature survey. A classical method is also included, because deep learning is not automatically superior on datasets of this size, and a deep model that cannot beat filter-bank common spatial patterns would not justify its complexity.

| Model | Architecture type | Key idea | Parameters, 875 samples | Parameters, 1,125 samples |
|---|---|---|---|---|
| **EEGNet (2018)** | Compact CNN | Depthwise and separable convolution | 3,188 | 3,700 |
| **ATCNet (2023)** | CNN with attention and TCN | Sliding-window attention over temporal features | 113,732 | 113,732 |
| **EEG Conformer (2023)** | CNN with Transformer | Convolution followed by global self-attention | 697,412 | 871,492 |
| **CTNet (2024)** | CNN with Transformer | EEGNet front end with a six-layer encoder | 152,364 | 153,004 |
| **FBCSP with LDA** | Classical | Filter-bank common spatial patterns | Not applicable | Not applicable |
| **HCT-Net (proposed)** | CNN with windowed attention | Combination of the components above | 20,996 | — |

*Table 15.1 — Baseline models and the proposed model. Parameter counts are for 22 channels and four classes, from the braindecode 1.8.1 reference implementations, at this project's 875-sample input and at the 1,125-sample input of the ATCNet reference implementation. The 2,548 often quoted for EEGNet corresponds to a window of about 2.2 s. Published accuracies are given separately, in Table 15.2 for cross-subject evaluation and Table 15.3 for within-subject evaluation, because they come from different protocols.*

| Model | A01 | A02 | A03 | A04 | A05 | A06 | A07 | A08 | A09 | Mean ± std | Kappa |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **ShallowConvNet+** | 66.84 | 46.53 | 67.53 | 52.26 | 34.38 | 39.76 | 65.45 | 71.18 | 66.84 | 56.75 ± 13.77 | 0.4234 |
| **DeepConvNet+** | 68.58 | 47.40 | 78.99 | 52.26 | 50.87 | 41.84 | 69.44 | 71.70 | 60.24 | 60.15 ± 12.71 | 0.4686 |
| **EEGNet+** | 69.79 | 42.01 | 79.51 | 50.87 | 35.76 | 37.15 | 65.80 | 67.36 | 63.37 | 56.85 ± 15.82 | 0.4246 |
| **EEG Conformer+** | 68.75 | 37.33 | 69.62 | 43.58 | 29.51 | 35.24 | 58.33 | 74.48 | 63.89 | 53.41 ± 17.08 | 0.3789 |
| **CTNet** | 69.27 | 43.92 | 79.34 | 55.38 | 43.92 | 36.11 | 65.10 | 70.66 | 64.06 | 58.64 ± 14.61 | 0.4486 |

*Table 15.2 — Cross-subject reference: leave-one-subject-out accuracy (%) on BCI IV-2a as reported by Zhao et al. (2024) [3], the CTNet paper. Each subject is held out in turn and the model is trained on the data of the other eight, for 600 epochs with batch size 512 and learning rate 0.001, on the 2–6 s window (1,000 samples) with z-score normalisation and no alignment; the held-out subject is used only for testing, with no fine-tuning. Models marked + were re-implemented by the CTNet authors under the same conditions. This is the only published comparison of these architectures under one cross-subject protocol that could be verified, and it is the reference for this project's cross-subject results.*

| Model | Accuracy | Source | Evaluation behind the figure |
|---|---|---|---|
| **CTNet** | 82.52% | Zhao et al. (2024) | Session 1 trains, session 2 tests; 30% of the training data held out for validation; 1,000 epochs |
| **EEG Conformer** | 78.66% | Song et al. (2023), authors' code repository | Hold-out, as stated in the repository |
| **ATCNet** | 85.38% | Altaheri et al. (2023), paper | Subject-dependent, as stated in the paper's abstract |
| **ATCNet** | 81.10% | ATCNet authors' code repository | Train–validation–test script: session 1 trains, session 2 tests |
| **EEGNet** | 68.67% | ATCNet authors' code repository | The same train–validation–test script as the 81.10% ATCNet figure |
| **EEGNet+** | 77.39% | Zhao et al. (2024), re-implemented | The CTNet paper's subject-specific protocol, as for CTNet above |
| **EEG Conformer+** | 77.66% | Zhao et al. (2024), re-implemented | The CTNet paper's subject-specific protocol, as for CTNet above |
| **FBCSP with LDA** | Approximately 68% | BCI Competition IV results (Ang et al.) | Competition protocol, session 1 trains, session 2 tests; not re-checked in the 2026 audit |

*Table 15.3 — Published within-subject accuracies on BCI IV-2a, with the source and the evaluation behind each. These figures come from different protocols and are comparable neither with each other nor with the cross-subject results in Table 15.2. An EEGNet figure of 71.50% that appeared in an earlier version of this table has been removed because no source for it could be verified.*

| Model | Main strength | Main limitation | Role in our experiments |
|---|---|---|---|
| **EEGNet** | Very small and trains reliably on limited data | Limited ability to model long-range temporal structure | Lower bound on model size; reference point for parameter efficiency |
| **ATCNet** | Attention applied locally, which suits short EEG trials | Considerably larger than EEGNet | Source of the windowed attention formulation used in the proposed model |
| **EEG Conformer** | Global self-attention with an interpretability method | Global attention is costly and needs augmentation to train well | Reference for the convolution-then-attention arrangement |
| **CTNet** | Strong within-subject accuracy (82.52%) | Reports a large drop under subject-independent evaluation | Provides the quantitative statement of the research gap |
| **FBCSP with LDA** | Well understood, fast, no training instability | Fixed feature extraction, no learned representation | Check that the deep models justify their additional complexity |

*Table 15.4 — Strengths, limitations, and the purpose each baseline serves in this project.*

### Why the published figures cannot simply be ranked

The within-subject accuracies in Table 15.3 were obtained under different evaluation protocols and cannot be ranked against one another. The same model on the same dataset differs by several points depending on who evaluated it: EEGNet is 68.67 per cent in the ATCNet repository's evaluation and 77.39 per cent in the CTNet paper's re-implementation.

Cross-subject figures are scarcer. Table 15.2 is the only published comparison of these architectures under one leave-one-subject-out protocol that could be verified, and it serves as the reference for this project's cross-subject results. This project's own protocol differs from it in stated ways — seven training subjects plus a separate validation subject, early stopping, a 3.5 s window and Euclidean alignment — so the comparison is indicative rather than exact.

Re-implementing all four under one identical leave-one-subject-out protocol is therefore a necessary part of this project, and the resulting comparison is useful independently of how the proposed model performs.

## 16. Evaluation Metrics

| Metric | Definition | Why it is used |
|---|---|---|
| **Accuracy** | Proportion of trials classified correctly | The headline measure, reported as a mean and standard deviation across the nine folds. Meaningful here because the classes are exactly balanced |
| **Cohen's kappa** | Agreement corrected for the agreement expected by chance | Standard in brain–computer interface research and directly comparable with published work. For a balanced four-class task, kappa of 0 corresponds to chance and 1 to perfect agreement |
| **Precision (macro)** | Of the trials predicted as a class, the proportion that belong to it, averaged over classes | Shows whether a class is being over-predicted |
| **Recall (macro)** | Of the trials belonging to a class, the proportion correctly identified, averaged over classes | Shows whether a class is being missed |
| **F1-score (macro)** | Harmonic mean of precision and recall, averaged over classes | Summarises per-class balance in a single number, so a model that ignores a hard class cannot hide behind the average |
| **Confusion matrix** | Counts of predicted against true classes | Identifies which classes are confused; feet and tongue are a known difficult pair for the reason given in Section 8.5 |
| **Parameter count** | Number of trainable parameters | Required for the comparison of model complexity, and for checking the budget in Table 11.3 |
| **Training time per fold** | Wall-clock time for one training run | Determines whether the experiment schedule is feasible |
| **Inference time per trial** | Wall-clock time for a single forward pass | Relevant to whether the model could run in a real-time setting |
| **Wilcoxon signed-rank test** | Paired non-parametric test across the nine subjects | Determines whether a difference between two models is statistically meaningful rather than fold-to-fold variation |

*Table 16.1 — Evaluation metrics. Statistical tests are applied at a significance level of 0.05.*

Results are reported for each subject individually as well as averaged. This matters because cross-subject decoders often fail badly on one or two particular subjects, a phenomenon sometimes described as brain–computer interface illiteracy. A model with a slightly lower mean accuracy but a smaller spread across subjects may be preferable in practice, and this distinction is invisible if only the average is reported.

# 17. Experimental Plan

## 17.1 Variables and controls

Stating what varies and what is held constant makes it possible to attribute any observed difference to a cause. In this project the independent variable is the model, and everything else is held fixed.

| Role | Item | Treatment |
|---|---|---|
| **Independent variable** | Model architecture | Varied: six models plus five component-study variants |
| **Independent variable** | Encoder depth (experiment E5 only) | Varied: 2 layers against 6 |
| **Dependent variables** | Accuracy, kappa, precision, recall, F1, parameters, time | Measured on every fold |
| **Controlled** | Dataset and preprocessing | Identical cached tensors for every model |
| **Controlled** | Fold membership | The same nine train / validation / test splits for every model |
| **Controlled** | Optimiser, schedule, batch size, stopping rule | Identical, as specified in Table 13.1 |
| **Controlled** | Random seeds | The same three seeds for every configuration |
| **Nuisance** | Subject-to-subject variation in signal quality | Handled by pairing: the Wilcoxon test compares models on the same subjects |
| **Nuisance** | GPU non-determinism | Reduced by seeding; residual effect quantified by the three-seed average |

*Table 17.1 — Variables and controls.*

## 17.2 Experiment definitions

| ID | Experiment | Configuration | Purpose | Runs |
|---|---|---|---|---|
| **E1** | Baseline reproduction | Each baseline, within subject, session 1 to session 2 | Confirm the implementations are correct before cross-subject results are trusted | 5 × 9 = 45 |
| **E2** | Cross-subject baselines | Each baseline under leave-one-subject-out | Establish a fair comparison under one protocol | 5 × 9 = 45 |
| **E3** | Proposed model | HCT-Net under leave-one-subject-out | Measure cross-subject performance of the proposed model | 9 |
| **E4** | Component study | Five variants, each removing one component | Identify which parts contribute to cross-subject performance | 5 × 9 = 45 |
| **E5** | Attention depth | Two encoder layers compared with six | Test the claim made in Section 11.6 | 9 |
| **E6** | Complexity comparison | Parameter count, training and inference time | Support the claim that the proposed model is comparatively lightweight | Measured during E2 and E3 |

*Table 17.2 — Planned experiments. The total is 153 training runs for one seed; every configuration is repeated with three seeds (Table 13.1).*

| Variant | Convolution | Windowed attention | Euclidean alignment | Augmentation |
|---|---|---|---|---|
| **V0 — full model** | Yes | Yes | Yes | Yes |
| **V1 — no attention** | Yes | Removed | Yes | Yes |
| **V2 — global attention** | Yes | Replaced by global | Yes | Yes |
| **V3 — no alignment** | Yes | Yes | Removed | Yes |
| **V4 — no augmentation** | Yes | Yes | Yes | Removed |
| **V5 — convolution only** | Yes | Removed | Removed | Removed |

*Table 17.3 — Component study configurations. Comparing V0 with V1 isolates the contribution of attention; V0 with V3 isolates alignment; V0 with V5 gives the total contribution of everything added to the convolutional core.*

## 17.3 Result reporting templates

The tables that the experiments will fill in are defined now, before any result exists. Fixing the reporting format in advance means the results cannot be presented selectively after the fact.

| Model | A01 | A02 | … | A09 | Mean | Std | Kappa | p vs. proposed |
|---|---|---|---|---|---|---|---|---|
| **FBCSP + LDA** | — | — | — | — | — | — | — | — |
| **EEGNet** | — | — | — | — | — | — | — | — |
| **ATCNet** | — | — | — | — | — | — | — | — |
| **EEG Conformer** | — | — | — | — | — | — | — | — |
| **CTNet** | — | — | — | — | — | — | — | — |
| **HCT-Net** | — | — | — | — | — | — | — | — |

*Template 17.1 — Cross-subject accuracy per subject. This is the principal results table for the 2nd Review.*

| Model | Accuracy | Kappa | Precision | Recall | F1 | Parameters | Train time / fold |
|---|---|---|---|---|---|---|---|
| **EEGNet** | — | — | — | — | — | 3,188 | — |
| **ATCNet** | — | — | — | — | — | 113,732 | — |
| **EEG Conformer** | — | — | — | — | — | 697,412 | — |
| **CTNet** | — | — | — | — | — | 152,364 | — |
| **HCT-Net** | — | — | — | — | — | 20,996 | — |

*Template 17.2 — Summary of metrics and cost. The parameter column is already known (braindecode 1.8.1 reference implementations at this project's 875-sample input) and is filled in; everything else is measured.*

| Variant | Mean accuracy | Change from V0 | Significant? | Interpretation |
|---|---|---|---|---|
| **V0 — full model** | — | reference | — | — |
| **V1 — no attention** | — | — | — | — |
| **V2 — global attention** | — | — | — | — |
| **V3 — no alignment** | — | — | — | — |
| **V4 — no augmentation** | — | — | — | — |
| **V5 — convolution only** | — | — | — | — |

*Template 17.3 — Component study. The interpretation column is written after the numbers are in, not before.*

### 17.4 Statistical procedure

1. For each pair of models, collect the nine paired per-subject accuracies.
2. Apply the Wilcoxon signed-rank test to the paired differences, two-sided, at a significance level of 0.05.
3. Where the proposed model is compared against all five baselines, apply a Holm–Bonferroni correction [14] across the five comparisons and report both the corrected and uncorrected values.
4. Report the effect size alongside the p-value, since with nine samples a non-significant result may still indicate a practically meaningful difference, and a significant one may not.
5. State explicitly when a comparison is inconclusive rather than describing it as a trend.

# 18. Tools, Frameworks and Environment

## 18.1 Software stack

| Category | Tool and version | Purpose |
|---|---|---|
| **Language** | Python 3.10 | All implementation |
| **Deep learning** | PyTorch 2.x with CUDA | Model definition, automatic differentiation, training |
| **EEG processing** | MNE-Python 1.6 or later [11] | Loading, filtering, epoching, topographic plotting |
| **Dataset access** | MOABB 1.0 or later | Programmatic download and standard dataset handling |
| **Reference models** | Braindecode 0.8 or later | Validated implementations of EEGNet, ShallowConvNet and related models |
| **Classical baseline** | scikit-learn 1.4, pyRiemann 0.6 | Filter-bank common spatial patterns with linear discriminant analysis |
| **Numerical computing** | NumPy 1.26, SciPy 1.11 | Filtering, alignment matrices, statistical tests |
| **Visualisation** | Matplotlib 3.8, Seaborn 0.13 | Figures, confusion matrices, topographic maps |
| **Experiment tracking** | TensorBoard, or Weights and Biases | Logging every run in Table 17.2 with per-fold metrics |
| **Configuration** | PyYAML | One configuration file per experiment |
| **Version control** | Git with GitHub | Code, configuration files and result tables |

*Table 18.1 — Software stack. Versions are recorded so that the environment can be reconstructed.*

Published reference implementations are used for the baselines wherever they are available. If a baseline were re-implemented from scratch and failed to reproduce its published accuracy, it would not be possible to distinguish an implementation error from a genuine difference in evaluation, and there is not enough time in Phase 3 to resolve such a question for four architectures.

## 18.2 Environment setup

**Reproducible environment**

```
# create and activate an isolated environment
python3.10 -m venv .venv
source .venv/bin/activate

# core scientific stack
pip install numpy==1.26.* scipy==1.11.* pandas matplotlib seaborn

# deep learning (CUDA build selected to match the driver)
pip install torch torchvision --index-url \
    https://download.pytorch.org/whl/cu121

# EEG-specific
pip install mne moabb braindecode pyriemann scikit-learn

# tracking and configuration
pip install tensorboard pyyaml

# freeze for reproducibility
pip freeze > requirements.lock
```

The locked requirements file is committed to the repository, so that the exact environment can be recreated. On hosted services such as Colab or Kaggle, the same file is used to install the environment at the start of each session.

## 18.3 Repository structure

**Project layout**

```
mi-eeg-cross-subject/
|-- configs/               one YAML file per experiment
|   |-- e1_reproduction.yaml
|   |-- e2_baselines_loso.yaml
|   |-- e3_hctnet_loso.yaml
|   |-- e4_component_study.yaml
|   '-- e5_attention_depth.yaml
|-- data/
|   |-- loaders.py         dataset download and channel selection
|   '-- splits.py          LOSO fold definitions
|-- preprocessing/
|   |-- filters.py         band-pass, epoching, rejection, scaling
|   |-- align.py           Euclidean alignment (Section 9.2)
|   '-- augment.py         segmentation and reconstruction
|-- models/
|   |-- eegnet.py  atcnet.py  ctnet.py  conformer.py
|   |-- fbcsp.py
|   '-- hctnet.py          the proposed model (Section 11)
|-- training/
|   |-- trainer.py         one fold, one model (Section 12.6)
|   '-- schedules.py
|-- evaluation/
|   |-- loso.py            the LOSO loop (Section 12.7)
|   |-- metrics.py         accuracy, kappa, precision, recall, F1
|   '-- stats.py           Wilcoxon, Holm-Bonferroni
|-- analysis/
|   |-- figures.py
|   '-- tables.py
|-- results/               per-fold metrics, logs (checkpoints ignored)
|-- tests/                 shape assertions, alignment check
|-- requirements.lock
'-- README.md
```

## 18.4 Reproducibility measures

- Every experiment is defined by a YAML configuration file committed to the repository, so any reported number can be regenerated by pointing the runner at its configuration.
- Every run records its seeds, the git commit hash, and the environment lock file alongside its results.
- Preprocessed tensors are cached with a hash of the preprocessing configuration in the filename, so a change to preprocessing cannot silently reuse stale data.
- Checkpoints are written every epoch to persistent storage, so an interrupted session costs one epoch rather than one fold.
- Unit tests cover the tensor shapes of every model and the alignment check described in Section 9.2; these run before any experiment.

# 19. Implementation Plan and Milestones

Phase 3 provides an eighteen-day implementation window. This section sets out the order in which the work is carried out, the reasoning behind that order, and the milestones that mark progress through it. Each milestone carries a completion criterion that can be checked objectively, so that progress is assessed against evidence rather than impression.

## 19.1 Sequencing logic

The plan is front-loaded deliberately. The data pipeline and the baselines are built and verified before the proposed model is written. The reason is that a result produced by a new model running on an unverified pipeline cannot be interpreted: if the accuracy is low, there is no way to tell whether the model is weak or the data are being handled incorrectly, and diagnosing that late in the window would leave no time to fix it.

A second consideration shapes the order. The stages differ greatly in how much they can be compressed if they run late. Writing code can be accelerated by working longer hours; training runs cannot, because they take as long as they take. The work is therefore arranged so that the compute-bound stages fall towards the end of the window with as few unresolved questions in front of them as possible, and so that the implementation stages, which are the ones that can absorb pressure, come first.

Three ordering rules follow from this and are not negotiable during Phase 3.

1. No model is trained on cached data until the alignment check described in Section 9.2 passes and the exploratory analyses in Table 10.1 match their expectations.
2. The proposed model is not written until at least two baselines reproduce their published within-subject accuracy.
3. No leave-one-subject-out experiment begins until the fold definitions have been checked, because an error there would invalidate every run that follows it.

A useful property of this arrangement is that implementing the proposed model does not depend on the baselines being finished. It depends only on the preprocessing being complete, so the model can be written and unit-tested while baseline training runs are still occupying the GPU. This overlap is the main source of slack in an otherwise tight schedule.

## 19.2 Timeline

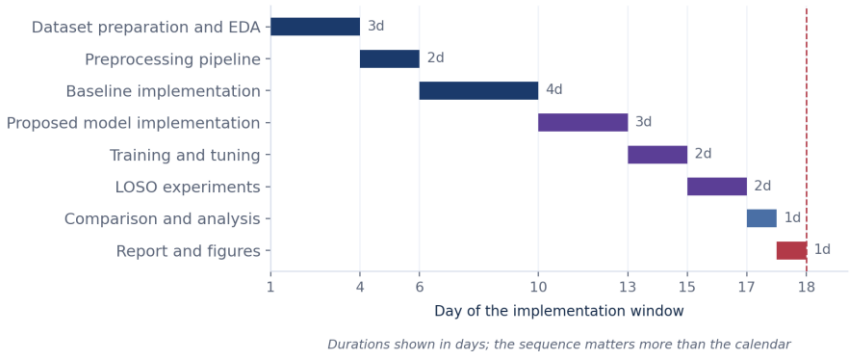



*Figure 19.1 — Sequence and relative duration of the implementation stages across the eighteen-day window.*

## 19.3 Milestone specification

Eight milestones are defined. Each is expressed as a condition that is either satisfied or not, rather than as an amount of work done, so that a milestone cannot be recorded as complete while an unresolved problem remains inside it.

| ID | Milestone | Definition of done |
|---|---|---|
| **M1** | Dataset prepared and verified | All nine subjects load successfully; trial counts, class balance, channel count and sampling rate match Table 8.2; the exploratory figures have been produced and reviewed against the expectations in Table 10.1 |
| **M2** | Preprocessing pipeline complete | Filtering, epoching, artefact rejection, alignment and caching are implemented; the average spatial covariance after alignment matches the identity matrix within tolerance; the rejection rate is recorded for each subject; cached tensors are written once and reused thereafter |
| **M3** | Baselines implemented and reproduced | All five baselines run end to end; within-subject accuracy falls within roughly three percentage points of the published values, or any difference is documented with an explanation |
| **M4** | Proposed model implemented | Tensor shapes match Table 11.2 at every layer; the parameter count is within five per cent of 20,996; the model reaches near-zero loss when deliberately overfitted on a single batch |
| **M5** | Training configuration fixed | Hyperparameters have been chosen using the validation subject only; configuration files are committed; a smoke test across two folds completes without error; checkpointing is confirmed to resume correctly |
| **M6** | Cross-subject experiments complete | Experiments E2 and E3 are finished for every model across all nine folds with three seeds each; per-fold metrics are written to the results directory |
| **M7** | Component study and analysis complete | Experiments E4 and E5 are finished; metrics, confusion matrices and statistical tests are computed; the reporting templates in Section 17.3 are populated |
| **M8** | Report ready for the 2nd Review | All tables and figures are produced; the findings are written up including any negative or inconclusive result; the repository is tidy and documented |

*Table 19.1 — Milestones and their completion criteria.*

The milestones are not of equal size. M1 and M2 are short but block everything that follows, so a delay there is the most expensive kind. M3 is the longest implementation block and also the one most likely to reveal an unexpected problem, since reproducing four published architectures is where discrepancies surface. M6 is the longest block of compute and has the least flexibility, because the number of runs is fixed by the experimental design rather than by choice.

## 20. Expected Outcomes

The following are the outcomes the project is designed to produce. They are stated as expectations. No experiments have been conducted at the time of writing, and none of these statements should be read as a result.

| Expected outcome | How it will be assessed |
|---|---|
| **A working implementation of five baseline models and the proposed model under one common pipeline** | Code in a version-controlled repository with recorded configurations |
| **A fair comparison of all models under one identical leave-one-subject-out protocol** | Template 17.1, with per-subject and averaged figures |
| **Cross-subject accuracy from the proposed model that is competitive with the baselines** | Mean accuracy and Cohen's kappa across nine folds, with a paired statistical test |
| **A parameter count lower than the attention-based baselines** | Direct measurement against the calculation in Table 11.3 |
| **Evidence of which components affect cross-subject performance** | Template 17.3, from the component study |
| **A clear statement of whether the proposed combination helps** | Reported whether the outcome is positive, negative or inconclusive |

*Table 20.1 — Expected outcomes and how each will be assessed.*

It is possible that the proposed model will not outperform the baselines under cross-subject evaluation. If that occurs, the component study will still indicate which parts of the design were responsible, and that finding is a legitimate contribution. The project is structured so that a negative result remains informative, which is why the component study is treated as a core experiment rather than an optional extension.

## 21. Possible Challenges and Solutions

| Challenge | Why it may occur | Planned response |
|---|---|---|
| **Limited training data** | 5,184 trials in total, of which roughly 4,032 are available for training in any fold | Augmentation, dropout, weight constraints, early stopping, and a deliberately small model |
| **Overfitting to training subjects** | The model can fit subject-specific detail that does not transfer | Validation on a held-out subject rather than a random trial split; capacity limited by design |
| **Baselines not reproducing published accuracy** | Differences in preprocessing or evaluation protocol between papers | Use published reference implementations; report both reproduced and published figures and explain the difference |
| **High variance across folds** | Only nine subjects, some of whom produce weak sensorimotor rhythms | Report per-subject results and standard deviation; use a paired test rather than comparing means alone |
| **Training time exceeding available GPU hours** | 153 runs per seed across all experiments, each repeated with three seeds | Cache preprocessed tensors, use mixed precision, reduce the maximum epoch count, and checkpoint every epoch so that an interruption costs one epoch rather than one fold |
| **Session interruptions on hosted GPU services** | Time limits on free tiers | Checkpoint to persistent storage so an interruption costs one epoch rather than one fold |
| **The proposed model does not improve on the baselines** | The combination may not help on this dataset | Report the result honestly and use the component study to explain it |
| **Attention stage unstable during training** | Small model dimension and a short sequence can make attention weights collapse | Monitor attention entropy during the smoke test; fall back to a single encoder layer if instability is observed, and record the change |

*Table 21.1 — Anticipated challenges and the response decided in advance for each.*

## 22. Novelty and Expected Contribution

This section states what is new in the project and what is not, so that the contribution is not overstated.

### 22.1 What is not new

The individual components of the proposed model are all taken from published work. Depthwise and separable convolution come from EEGNet, sliding-window attention comes from ATCNet, the arrangement of convolution followed by a Transformer encoder comes from CTNet and EEG Conformer, Euclidean alignment comes from the transfer learning literature [7], and segmentation-and-reconstruction augmentation comes from EEG Conformer and CTNet. Leave-one-subject-out evaluation is a standard protocol. None of these is presented as an original contribution.

### 22.2 What the project contributes

- A comparison of EEGNet, ATCNet, CTNet and EEG Conformer under one identical subject-independent protocol. The published figures for these models were obtained under different evaluation settings and cannot be ranked against one another as they stand.
- A specific lightweight combination of these components, at 20,996 parameters, with the attention depth reduced to suit the size of the dataset, and evaluated for cross-subject rather than within-subject performance.
- A component study that measures how much each part of the combination contributes to cross-subject accuracy, which the surveyed papers do not report.
- A comparison of accuracy against parameter count under a single protocol, which supports a practical judgement about whether the additional complexity of attention-based models is justified on a dataset of this size.

The contribution is therefore a careful comparative study together with a concrete lightweight design, rather than a claim of a fundamentally new architecture. This is stated deliberately, and no claim of improved performance is made in advance of the experiments.

## 23. Future Scope

- Extending the evaluation to BCI Competition IV Dataset 2b and to larger collections such as the PhysioNet motor imagery dataset, to test whether the findings hold with more subjects and different electrode configurations.
- Measuring how many labelled trials from a new user are required to reach a given accuracy, which would quantify the calibration effort a real deployment would need.
- Investigating domain adaptation methods, including the adversarial variant described in Section 13.2 and contrastive approaches, as a more systematic way of reducing the difference between subjects.
- Applying interpretability methods such as class activation topography to check whether the model relies on the expected sensorimotor regions rather than on artefacts.
- Testing the model in an online setting, where trials arrive continuously and a decision must be produced within a fixed latency budget.
- Reducing the model further through quantisation or pruning, with a view to running it on embedded hardware suitable for a wearable device.

## 24. References

[1] V. J. Lawhern, A. J. Solon, N. R. Waytowich, S. M. Gordon, C. P. Hung and B. J. Lance, “EEGNet: A Compact Convolutional Neural Network for EEG-based Brain–Computer Interfaces,” Journal of Neural Engineering, vol. 15, no. 5, 056013, 2018.

[2] H. Altaheri, G. Muhammad and M. Alsulaiman, “Physics-Informed Attention Temporal Convolutional Network for EEG-Based Motor Imagery Classification,” IEEE Transactions on Industrial Informatics, vol. 19, no. 2, pp. 2249–2258, 2023.

[3] W. Zhao, X. Jiang, B. Zhang, S. Xiao and S. Weng, “CTNet: A Convolutional Transformer Network for EEG-Based Motor Imagery Classification,” Scientific Reports, vol. 14, 20237, 2024.

[4] Y. Song, Q. Zheng, B. Liu and X. Gao, “EEG Conformer: Convolutional Transformer for EEG Decoding and Visualization,” IEEE Transactions on Neural Systems and Rehabilitation Engineering, vol. 31, pp. 710–719, 2023.

[5] M. Tangermann, K.-R. Müller, A. Aertsen et al., “Review of the BCI Competition IV,” Frontiers in Neuroscience, vol. 6, 55, 2012.

[6] C. Brunner, R. Leeb, G. Müller-Putz, A. Schlögl and G. Pfurtscheller, “BCI Competition 2008 – Graz Data Set A,” Institute for Knowledge Discovery, Graz University of Technology, 2008.

[7] H. He and D. Wu, “Transfer Learning for Brain–Computer Interfaces: A Euclidean Space Data Alignment Approach,” IEEE Transactions on Biomedical Engineering, vol. 67, no. 2, pp. 399–410, 2020.

[8] R. T. Schirrmeister, J. T. Springenberg, L. D. J. Fiederer et al., “Deep Learning with Convolutional Neural Networks for EEG Decoding and Visualization,” Human Brain Mapping, vol. 38, no. 11, pp. 5391–5420, 2017.

[9] K. K. Ang, Z. Y. Chin, H. Zhang and C. Guan, “Filter Bank Common Spatial Pattern (FBCSP) in Brain–Computer Interface,” Proceedings of the IEEE International Joint Conference on Neural Networks, pp. 2390–2397, 2008.

[10] A. Vaswani, N. Shazeer, N. Parmar et al., “Attention Is All You Need,” Advances in Neural Information Processing Systems 30, 2017.

[11] A. Gramfort, M. Luessi, E. Larson et al., “MEG and EEG Data Analysis with MNE-Python,” Frontiers in Neuroscience, vol. 7, 267, 2013.

[12] B. Aristimunha, I. Carrara, P. Guetschel et al., “Mother of All BCI Benchmarks (MOABB),” software framework for reproducible EEG decoding, 2023.

[13] G. Pfurtscheller and F. H. Lopes da Silva, “Event-Related EEG/MEG Synchronization and Desynchronization: Basic Principles,” Clinical Neurophysiology, vol. 110, no. 11, pp. 1842–1857, 1999.

[14] S. Holm, “A Simple Sequentially Rejective Multiple Test Procedure,” Scandinavian Journal of Statistics, vol. 6, no. 2, pp. 65–70, 1979.

[15] F. Wilcoxon, “Individual Comparisons by Ranking Methods,” Biometrics Bulletin, vol. 1, no. 6, pp. 80–83, 1945.